Journal of Intelligent & Fuzzy Systems 36 (2019) 6181–6194
DOI:10.3233/JIFS-181998
IOS Press

Version specific test case prioritization approach based on artificial neural network

Hosney Jahan[a], Ziliang Feng[a,*], S.M. Hasan Mahmud[b] and Penglin Dong[a]
[a]*College of Computer Science, Sichuan University, Chengdu, China*
[b]*School of Computer Science and Engineering, University of Electronic Science and Technology of China, Chengdu, China*

Abstract. Regression testing involves validating a software system after modification to ensure that the previous bugs have been fixed and no new error has been raised. Finding faults early and increasing the fault detection rate are the main objectives of regression testing. A common technique involves re-executing the whole test suite, which is time consuming. Test case prioritization aims to schedule the test cases in an order that could achieve the regression testing goals early in the testing phase. Recently, machine learning techniques have been extensively used in regression testing to make it more effective and efficient. In this paper, we propose and investigate whether an Artificial Neural Network (ANN)-based approach can improve the version specific test case prioritization approach. The proposed approach utilizes the combination of test cases complexity information and software modification information with an ANN, for early detection of critical faults. Three new factors have been proposed, based on which an ANN is trained and finally it can automatically assign priorities to new test cases. The proposed approach is empirically evaluated with two software applications. Effectiveness metrics, such as fault detection rate, accuracy, precision, and recall are examined. The results suggest that the proposed approach is both effective and feasible.

Keywords: Regression testing, test case prioritization, artificial neural network, fault detection capability

1. Introduction

Regression testing is the process of retesting a software system in order to validate that the modified parts does not contain any error and the remaining portions are not affected due to the modification [1]. A common and traditional approach is re-executing all the test cases. However, this process makes the testing task costly and infeasible due to limited resources. To this end, regression testing provides systematic ways to re-execute all or some of the test cases of a previous version in order to verify the functionalities of the current version. A number of research and studies have been conducted on this area, and several techniques have been proposed, which includes: test case selection [2], minimization [3] and prioritization [4, 5]. The prior two techniques involve reduction of the test suites, mostly based on the software modification [6] and coverage information [7]. However, both these approaches involve a risk of eliminating fault revealing test cases [8]. On the other hand, test case prioritization orders test cases based on some criteria which enables the testers to execute the most important test cases early in the testing process, therefore, the testing cost will be reduced correspondingly.

In the past several decades, a number of test case prioritization approaches have been studied to order test cases and increase the likelihood of revealing faults early. Existing approaches include: model-based [1], history-based [9], and code-based [10]. Code-based approaches, more specifically version

*Corresponding author. Ziliang Feng, College of Computer Science, Sichuan University, Chengdu 610065, China. E-mail: fengziliang@scu.edu.cn.

specific prioritization approaches may reflect faults more explicitly as the test cases are prioritized with the knowledge of the modification. Since regression testing takes place for a particular version of a software, version specific approaches may play an important role in prioritizing test cases in a manner which will be most effective for that version.

Recently, machine learning-based approaches have been used in test case prioritization. The basic concept of applying these approaches involve creating a training dataset based on various features of the test cases. Afterwards, the test cases are labelled with several priority classes based on some criteria. A machine learning algorithm is then trained on the training dataset. After training it can classify new test cases according to their priority classes. Finally, the test cases predicted with highest priority are executed earlier, revealing the most important faults early in the testing phase.

In this paper, a machine learning-based supervised approach has been proposed in order to investigate whether it can bring promising advantages in terms of version specific prioritization. We concentrate on ordering test cases according to a priority value, for detecting faults or executing test cases covering defective modules early. The main objective of our technique is to order test cases based on a given set of training data so that it resembles to the insight of a tester while assigning priorities to new test cases. To achieve this goal, we employed an artificial neural network for our research to build a classifier based on given supervised training data. ANN has the ability to understand a highly complex system from its behaviour. Since, test cases also represent the behaviour patterns of a system in terms of various execution scenarios, it could be a fair idea to use ANN for prioritizing test cases.

Test cases are usually prioritized based on various factors or artifacts, such as cost-awareness [5], coverage information [10], requirements [11], and specification [12] etc. In this research, mainly three factors have been proposed for the prioritization, which are: number of modified modules (MM), test cases complexity (TC) and number of changed requirements (CR). All these features are related to a particular version of a system under test (SUT). The features are extracted for each version of the SUT. An artificial neural network is then used to learn the relationship between the input features and outputs priorities of new test cases as exercised. The hypothesis behind considering these factors is that the test cases which cover a number of modified modules and changed requirements are more likely to find faults, since faults are mostly arise from the modifications.

Moreover, the proposed approach provides a test case selection strategy, which selects only the test cases related to the modification and reduces the size of the test suite correspondingly.

In this paper, the following contributions have been made:

- An approach for version specific test case prioritization based on three factors for early fault detection.
- A realization of our approach using ANN in order to obtain new test cases according to their priorities.
- A set of empirical studies to demonstrate the effectiveness of our approach in terms of fault detection rate and prediction accuracy compared to total statement coverage prioritization, re-test all (untreated) and random prioritization.
- A test case selection strategy to reduce the test suite size.

The rest of the paper is organized as follows. Section II briefly reviews the existing related work. Section III presents the proposed approach and the corresponding algorithms. Section IV presents the empirical evaluation of the proposal. The results and discussion with two software systems are presented in Section V. Finally, Section VI concludes the paper with a discussion of future work.

2. Related work

In this section, we present an overview of the state-of-the-art relevant approaches. The literature further divided into three subsections (2.1–2.3).

2.1. General test case prioritization approaches

The most important part of test case prioritization is the selection of proper attributes based on which the test cases are prioritized. We broadly classified previous attributes as coverage-based, requirement-based, history-based and cost-based approaches. Coverage-based approaches take into account the coverage information of a system such as statement coverage, method coverage [10, 13] etc. Daniel et al. [14] presented a case study on coverage-based regression testing techniques using a real world industrial system, including real faults and evaluated the results based on four regression testing approaches. James et al. [15] proposed a test case prioritization algorithm

based on the coverage information of the modified condition or decision. However, all these coverage-based approaches aims to achieve 100% coverage which may require additional cost and resources than their allocation.

Requirement-based approaches considers the requirement information of a system for prioritizing test cases. Since, test cases are generated in order to meet a system's requirements, these information could play important role in terms of detecting crucial faults. Krishnamoorthi et al. [11] proposed a test case prioritization approach which aims to detect severe faults. They proposed six factors to achieve this goal where three of them are considered for new test cases and others are considered for regression test cases. Srikanth et al. [16] extended their previous work by applying two prioritization factors in an enterprise level cloud application. Another approach is proposed by Charitha et al. [17], considering the risks associated with the requirements to detect faults related to the risky components. Most of these approaches, considered the number of times a requirement have been changed during the development phase, as an attribute. In contrast to these approaches, we used a factor (CR) which yields the total number of changed requirements in a test case.

History-based approaches [9, 18–20] have been widely used in test case prioritization. In these approaches, the previous historical data are analyzed and then related factors are determined for prioritization. Hyuncheol et al. [9] presented a history-based approach where they utilized the execution time and test criticality as the factors for prioritization. A black-box based prioritization approach is proposed by Bo Qu et al. [18] uses the test history and run-time information to compute test cases relationship matrix, and predict the fault detection relationship within the test cases for achieving better fault detection rate. Srikanth et al. [19] further reported a technique that prioritizes system use cases and combined them with the historical data to order test cases. Emelie et al. [20] applied several factors such as execution history, static priority, execution cost, test case age etc. to do the same.

Cost-based approaches [5, 21] are quite related to the history-based approaches since they also utilizes previous historical data in order to minimize the testing cost. However, all these techniques require previous historical data which may not be available for a system. Different to these approaches, our approach is solely based on the modifications of a system; hence, does not require any historical data.

2.2. Machine learning-based test case prioritization approaches

Machine learning-based prioritization approaches include clustering and classification methods for both white-box and black-box testing. Lenz et al. [22] formulated an approach that utilizes the test results of various testing techniques. Three clustering algorithms are being applied on the gathered data to produce equivalent classes. Later these equivalent classes are combined with previous data to train a decision tree algorithm which can be used for test case selection and prioritization. Nida Gökçe et al. [34] proposed a coverage-based prioritization approach using artificial neural networks. They firstly constructed a set of event sequence graphs (ESG) of the system under test. Afterwards, they applied a neural network clustering method to divide the dataset into several groups. Finally, the test cases are manually prioritized based on their significance given by several attributes of the relevant events of the ESG. Another requirement based clustering approach is reported in [24] using previous fault history.

Usually, test cases from the same cluster reveal similar faults since they share the same attributes, which may hinder the fault detection rate. To overcome this drawback, Abu Hasan et al. [25] proposed a dissimilarity-based clustering technique by combining test case's similarity information with the historical data. Though all these approaches showed good results, however, in clustering based approaches it is a dilemma to determine how many clusters should be constructed.

Remo Lachmann et al. [26] proposed a black-box based prioritization approach for manual testing by applying ranked support vector machine (SVM RANK). Test history and other high-level artifacts such as requirements and failure history are considered to perform the prioritization. Another black-box testing approach is presented by Bhasin [27] applying artificial neural networks. They constructed a Module State Diagram (MSD) based on the specification of the SUT and prioritized the module interactions, based on which test cases are finally prioritized. However, proper validation of the experiment is not provided. A framework has been introduced in [28] which combines an artificial neural network with the program slicing technique to prioritize test cases. Similar to our work they use the system modification information for prioritization.

However, we could not perform comparison with these classification techniques since the prior two

techniques [26, 27] are black-box based while our approach is white-box based (version specific). Moreover, the third approach [28] differs due to their used artifacts, for example, they used test trace history in addition to the modification information to construct the feature vectors, our approach, on the other hand, requires only the modification information.

2.3. Other approaches

Besides the aforementioned techniques, several other approaches have been frequently used for test case prioritization, such as, Bayesian Network [29], Fuzzy Logic [30], Genetic Algorithm [31], Ant Colony Optimization [32] and so forth. A Bayesian network based prioritization approach is introduced in [29] where the probability of finding bugs is calculated for each test case based on two proposed features: Feedback mechanism and modification information. Gary et al. [31] reported a black-box based prioritization technique using software specification. Mainly two factors (requirement severity score and inter-case dependency) have been considered as the basis for prioritization. Afterwards, Genetic Algorithm and Ant Colony Optimization techniques are applied to order the test cases.

Perini et al. [33] proposed a method for prioritizing requirements instead of test cases referred to as case-based ranking (CBRank). They utilized the information provided by the project's stakeholder and applied a machine learning based boosting algorithm to predict the priority values. In addition to these methods, though formal methods are widely used in the field of test case generation [34, 35], recently they are also being used in prioritization [1].

All these aforementioned methods differ from our approach due to their underlying concepts and used factors for prioritization. Moreover, none of them applied machine learning algorithms in terms of version specific prioritization. In this paper, we propose an artificial neural network based version specific prioritization method to find the most important test cases for a specific version, increasing the fault detection rate.

3. Proposed prioritization approach

This section presents the proposed test case prioritization approach and describes the algorithms used. Figure 1 illustrates the approach, demonstrating the systematic process employed to prioritize the test cases. The first step includes feature collection and pre-processing. The objective of this initial step is to extract features from the test cases.

To do so, we performed a textual analysis of the program's source code (applying the proposed algorithms) and gathered necessary data (test cases complexity information and software modification information). Since the proposed approach follows version specific test case prioritization process, we require only those test cases that are related to the current modified software version. Therefore, the modification related test cases are selected based on the information of the extracted features. Next, a supervised classification algorithm is applied, under the strategy of artificial neural network to predict the

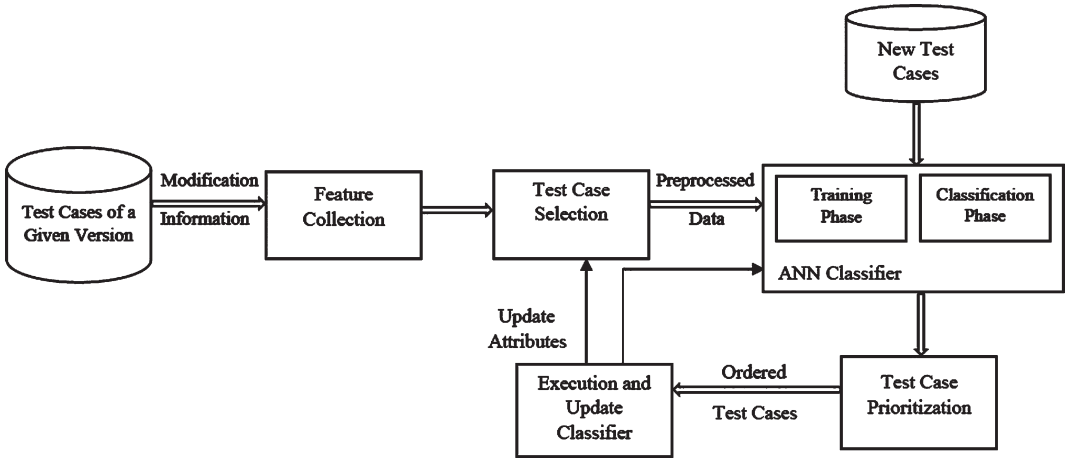

Fig. 1. Overview of the test case prioritization approach.

priorities of the test cases. The ANN model consists of two phases: training and classification. In the training phase, an ANN model is built and presented with a feature vector and a target vector, where the feature vector represents the input features and the target vector represents the corresponding class label of each training sample. The built ANN model discovers the hidden knowledge of the training samples and learns their relationship to prioritize the test cases. Meanwhile, the model performance is validated and tuned by applying the k-fold cross validation [36]. Finally, the trained ANN model operates on a set of unlabeled test cases and decides their priorities. In the last step, the test cases are executed according to their prioritized order for early fault detection.

3.1. Feature collection

In the proposed approach several features (factors) are mainly considered for the prioritization. Here, we introduce the features and the associated algorithms to extract them from the program source files.

3.1.1. Feature vector

Number of Modified Modules (MM). This feature denotes the total number of modified modules covered in a test case. In a software project, faults are mainly arise due to the modifications. Hence, the number of modified modules in a test case could be a good indicator to reveal faults i.e., a test case covering more modified modules is more likely to exhibit faults.

In order to extract this feature, a comparative analysis of the two versions (main system and the modified version) of the SUT is performed. From the comparison, the modified modules that are invoked by the test case code has been identified and counted for each test case. Note that, the term module is used here as the alternative of method or function. Algorithm 1 presents the computation of MM.

In addition, the sequence of covered modified modules for each test case are also recorded which are later used as another feature, referred to as **Modified Modules Sequence (MMS)**. The feature value for a modified module is either 0, if it is not covered by the test case or 1 otherwise.

Test Cases Complexity (TC). The complexity of a test case is represented by the number of requirements in a test case. As mentioned earlier, our test cases are function (method) level where a function may cover more than one requirement. Thus, a test case may cover more than one requirements as well. A test case covering many requirements is considered as more complex and vice versa.

Algorithm 1. Modified Modules Count

Input: System Under Test or SUT (S), Modified Version of the SUT (V) and Test Suite (T)

Output: Number of Modified Modules (MM)

Declare: C is a list of modified conditions, A1 and A2 are array lists, t_i is a single test case

1. begin
2. Initialization: C $\leftarrow \emptyset$, A1, A2 $\leftarrow \emptyset$
3. Load the source code files: S, V and T
4. Separate the methods of S by using the keyword public void, use the methods as strings, and add the strings to A1
5. Separate the methods of V by using the keyword public void, use the methods as strings, and add the strings to A2
6. Compare the elements of A1 and A2 (i.e. the methods of S and V)
7. if any condition mismatch then
8. Add the condition to C
9. Store the method name associated with the mismatched condition into a map
10. end if
11. for each test case $t_i \in T$ do
12. Find the method names in t_i
13. Count the number of methods in t_i which contain modified conditions, using step 9
14. end for
15. end

In order to get the complexity information, an analysis of the source code of the modified version is done. Algorithm 2 presents the computation steps of TC. The number of covered requirements in each of the methods are counted first. Since, a test case is a sequence of methods, for each test case, all the covered requirements by the associated methods are computed. Here, we consider the requirements regarding branch coverage following the previous work done by Chu-Ti Lin et al. [7]. They defined requirements of a program in terms of branch coverage, and denoted a true case and a false case for each of the if-statements (requirements). Though, they used this process for test suite reduction, here, we apply it for prioritization.

Number of Modified Requirements (MR). This feature presents the total number of changed requirements in a test case. Since, test cases exhibit a system's requirements, change in requirements may play a significant role in revealing faults. MR is collected by following a similar process to Algorithm 1 (comparing the main system and the modified version). Meanwhile, the modified conditions are marked and stored separately for each of the test

Algorithm 2. Test Cases Complexity Computation

Input: Modified Version of the SUT (V) and Test Suite (T)

Output: Test Cases Complexity (TC)

Declare: Count is a list of requirements, M is an array list, t_i is a single test case

```
1.  begin
2.  Initialization: Count ← ∅, M ← ∅,
3.  Load the source code files: V and T
4.  Separate the methods of V by using the keyword
    public void, use the methods as strings,
    and add the strings to M
5.  for each method m in M do
6.      if m contains any conditional statement
        then
7.          Add the condition to Count
8.          Store the method name in a map
9.      end if
10. end for
11. for each test case t_i ∈ T do
12.     Identify the method names in t_i
13.     Count the number of conditions for the associated
        methods of t_i using step 8
14. end for
15. end
```

cases, to be used as another feature, referred as **Modified Requirements Sequence (MRS)**. A modified requirement is represented as 1, if it is covered by a test case; the value is 0 otherwise. Other than these five features, our feature vector includes some other information from the test cases.

A test case can be represented as a triple (pre-condition, triggering event, post-condition) where, the pre-condition stands for the input data of an event, and post-condition represents the test verdict or expected result [34]. Thus, input data and expected result are the most important parts of a test case. We used these two features as a part of our feature vector.

3.1.2. Target vector

In order to decide which test case should be labeled with which priority, the factors MM and MR have been considered. We added the values of MM and MR for each test case and ordered them in descending order based on the result of the addition. Afterwards, the test cases are manually divided according to the number of priority classes. To decide the number of priority classes we performed some experiments with different numbers. The results of the experiments indicate that the number of priority classes may vary according to the size of the SUT. Since, we used two medium sized SUTs, only three priority classes were most suitable. The results of the experiments with other priority ranges, for example: 2 (less than 3) and 5 (greater than 3) shows that they struggle with the class imbalance problem [37]. In such a problem, the majority samples are labeled with a class, while far fewer other samples are labeled with other classes. This imbalance distribution of samples make the machine learning algorithms incapable of predicting the minority class, which greatly effects the prediction accuracy of the model. In order to avoid this problem, the training samples should be chosen in a way where the number of class elements are balanced. In our case, by choosing three priority classes we were able to avoid the class imbalance problem for both the SUTs. Therefore, we labeled the test cases by assigning a priority value from 0 to 2. Test cases labeled with priority 2 are considered as the most important ones and those labeled with priority 0 are considered as the less important.

However, for large sized test suites, classifying it into three groups may not be very useful. Therefore, we can increase the number of class levels based on the size of the SUT and test suite, as long as the class imbalance problem does not occur.

3.2. Test case selection

For the implementation of this step, we used the information extracted by the feature MM. The test cases for which the value of MM is at least 1 or more than 1 are added to a new test suite. The output of this phase establishes a test suite (T_{MM}) which consists only the test cases which are directly related to the software modification, as shown in Algorithm 3.

In practice, an ad-hoc based approach is usually followed for the selection and execution of test cases. Our approach, on the other hand, follows a systematic way for selecting test cases which further states the scope of the prioritization.

After the feature extraction and test case selection phase, all these information are transformed into a vector representation, i.e., a data set is constructed for all the test cases according to their attributes. The dataset is then fed to the ANN model to train it.

3.3. ANN classifier

The aim of our technique is to prioritize test cases by imitating the behavior of human testers. To this end, we employ artificial neural network to learn an ordered classification model based on the training dataset. Artificial neural networks [38] bear a resemblance to the structure and information processing abilities of the brain. It has been extensively adopted

Algorithm 3. Test Case Selection

Input: Original Test Suite (T)

Output: Test Suite of Selected (Modification Related) Test Cases (T_{MM})

Declare: MM is the number of modified modules in a test case, t_i is a single test case

```
1.  begin
2.  Initialization: T_MM ← ∅
3.  for each test case t_i ∈ T do
4.  if MM >= 1 then
5.      Add t_i to T_MM
6.      Remove t_i from T
7.  end if
8.  end for
9.  Return T_MM
10. end
```

and widely used in the area of pattern recognition, data mining, machine learning and bioinformatics. They are presented with a set of sample data and ascertain the concealed idea of the data by performing mathematical calculations on them.

ANNs consist of a collection of interconnected information processing units known as neurons. The neurons are usually organized in three layers: an input layer, one or multiple hidden layers, and an output layer. The input layer neurons passes the input signal to the first hidden layer via the synaptic weights. The hidden layer neurons calculates a weighted summation of the inputs and applies an activation function to determine whether to pass the value to the next layer. Thus, the learning process of the neural network continues by adjusting the weights. Back propagation algorithm is a commonly used approach for computing and adjusting the weights. In this research, we applied the gradient descent optimization algorithm which uses the concept of back propagation.

3.3.1. Training phase

The training phase further contains three sub steps: building model, pre-processing and implementation of training algorithm. The parameters to be considered for building the model includes: number of neurons in the input layer, number of hidden layers, number of neurons in each hidden layer, number of neurons in the output layer (for multi-class problem), activation function, number of epochs etc. The best parameters are chosen by tuning the parameters via grid search. The number of input units are chosen based on the inputs of the subject programs. The output layer consists of three units based on the three priority classes. Two hidden layers have been chosen each containing 20 units. For hidden layers, the activation function Relu is applied, while sigmoid function is used for the output layer.

The training sets need some pre-processing before starting the training process. Since, we are dealing with a multi-class classification problem (3 priority classes), each of the classes are encoded into a binary class matrix. Then, the input features are scaled to avoid the domination of one feature to another. We standardized the input feature vectors within the range of [0, 1] by applying Equation (1). Finally, the training algorithm has been implemented in Python language using Python 3.4 version and Keras library [39].

$$x_{stand} = \frac{x - mean(x)}{standard\ deviation(std)} \tag{1}$$

3.3.2. Classification phase

Once the classifier is trained, it can be applied for the prioritization of other test cases. The model is now able to predict priority for new test cases which are associated with the software modification. Thus, a huge number of test cases could be ranked automatically within a short period of time, increasing time effectiveness. Besides, the testing effort for creating appropriate test suite is also minimized since the tester now only has to classify a portion of the test cases used as training data.

3.4. Test case prioritization

The classification results received from the classifier are in random order by default, therefore, we listed them in preferred groups according to their priorities, where, test cases with same priority are listed in same group. As mentioned in subsection 3.1.2, three priority levels were most suitable for the SUTs we used for our experiments. Then, we arranged the test cases into three priority groups from 0 to 2, representing the most critical test cases as 2 and less critical ones as 0. Since, test cases of a priority group denotes the same priority, they can be ordered either sequentially or randomly in the group. Here, we sequentially ordered the test cases of same priority. Afterwards, the test cases from the three groups are included in a test suite in descending order. Note that, though we choose three priority groups for the SUTs, the number of chosen groups can be increased or decreased (subsection 3.1.2) based on the size of the SUT and number of test cases.

The proposed technique aims to minimize the testing effort in two ways. First, it reduces the effort for creating a prioritized test suite, since, once the classifier is trained it is reusable and is able to predict priorities for different test cases automatically, within a short time. On the other hand, performing manual prioritization or re-executing all the test cases are both time and cost consuming. Second, it increases the time effectiveness by finding failures earlier than the re-test all and random prioritization techniques. Besides, it offers the testers with a choice of preference while executing the test cases i.e. the tester may execute only the highest prioritized test cases in terms of time and resource restriction. Moreover, other classification algorithms are also applicable to our approach indicating its flexibility.

3.5. Execution and update classifier

After ordering and adding the test cases in a new test suite, they can be executed according to the preference of the tester i.e., he can execute only the test cases of highest priority, in case of limited resource and time. Once a classifier is trained, it is reusable as often as required. Since, a software goes through continuous evolution during its lifetime, the same classifier may not be suitable for all the evolved phases. For example, if a new feature is added to the software which the classifier is not familiar with, it may not provide correct prediction for the new test cases. Same situation can be raised for new or changed requirements. Even, in some cases, test cases can also be modified to verify the changes in a software. All these situations stand in need for a new or updated classifier i.e., a new classifier should be built or the previous one should be retrained with updated training data set, containing the latest modification information. Though, updating the training data set may require some additional task, it is still less tedious than manual prioritization and selection of test cases for each test set [26].

4. Validation techniques

The effectiveness of the proposed approach is evaluated based on two software applications with multiple versions, considering two aspects: 1. prediction performance of the classifier and 2. fault detection capability.

4.1. Evaluation metrics

This section presents a brief review of the metrics used in this research for evaluating the results.

4.1.1. Metrics to measure the prediction performance of the classifier

Most of the supervised machine learning based prioritization approaches [26–28] does not pay much attention on the classification quality, though, in machine learning, it is an important and common practice for gaining confidence on the classifier. Moreover, a trained classifier is reusable. Hence, it is necessary to evaluate its prediction performance for future usage.

To do so, three evaluation metrics, i.e., accuracy, precision and recall are calculated accordingly. Their corresponding calculating formulas are as follows:

$$Accuracy = \frac{TP + TN}{TP + FP + TN + FN} \tag{2}$$

$$Precision = \frac{TP}{TP + FP} \tag{3}$$

$$Recall = \frac{TP}{TP + FN} \tag{4}$$

Where, TP is the number of test cases that are correctly predicted for each priority class, TN denotes the number of test cases that are correctly rejected for certain priority class, FP stands for the number of test cases that are incorrectly predicted as certain priority (i.e., test cases of other priority classes incorrectly classified as a certain class), and FN represents the number of test cases that are incorrectly rejected for certain priority (i.e., test cases of a certain priority class incorrectly classified as other classes), respectively.

Here, **Accuracy** defines, how accurately the classifier classifies the test cases according to their priorities. **Precision** defines, for all the predicted test cases of a certain priority, how many of them have been correctly predicted. As an example, suppose, there are total 100 test cases that has been predicted as priority 2. Among them, 80 are correctly classified and remaining 20 from other priority classes are incorrectly classified as Priority 2. Therefore, Precision = 80 / (80+20) = 0.80. **Recall** defines, for all the test cases that should have a certain priority label, how many of them have been correctly classified. Ex: suppose, there are 100 test cases that should have a priority label 1. However, 30 of them has been

incorrectly classified as priority 2 and 0, and other 70 are correctly classified. Therefore, Recall = 70 / (70+30) = 0.70.

In addition to these metrics, we further estimate the required time to train the ANN model and classify the results in order to determine the applicability of the proposed approach in practice.

4.1.2. Metrics to measure the fault detection capability of the proposed approach

To evaluate the effectiveness of the prioritization, the Average Percentage of Faults Detected (APFD) metric, proposed in literature [10] has been considered. The APFD metric computes the average fault detection rate of a prioritized test suite within the range of 0 and 1, where higher values indicate better fault detection capability and vice versa. The computation formula of APFD is as follows:

$$APFD = 1 - \frac{TF_1 + TF_2 + \ldots + TF_m}{n \times m} + \frac{1}{2n} \quad (5)$$

Where, n is the total number of test cases in the test suite, m denotes the number of faults in the SUT and TF_f represents the position of the test case in the prioritized test suite that detects fault f.

We further compute the test suite reduction rate (TR) by using Equation (6).

$$TR = \frac{T - T_{MM}}{T} \times 100 \quad (6)$$

Where, T denotes the total number of test cases in the original test suite and T_{MM} denotes only the test cases that covers at least one or more modified modules.

4.2. Experimental subjects

For evaluation purpose, the proposed approach was applied on two medium-sized software applications with multiple versions.

Case Study 1: Credit Card Approval System. The first case study is used from literature [40] where it was used to evaluate an ANN based test oracle. The system accepts or rejects a credit card approval application based on several input combinations. During the implementation, few more requirements are added in the system, containing a total of 48 requirements and 416 test cases. Three versions of the subject system have been implemented each containing a number of faults. Version 1 contains 3 faults, version 2 contains 4 faults and version 3 contains 5 faults, respectively. According to different modification information of the three versions, input vector is fed to the ANN model by 21, 21 and 23 inputs, respectively. The target vector includes 3 outputs for all the versions.

Case Study 2: Registration-Verifier Application. The second case study is provided by the authors of [41]. This software maintains and manages the student's records and validates their registration based on the Iranian Universities Bachelor-Students Registration Policies. The subject system consists of 70 requirements and 523 test cases. Two different versions have been implemented where version 1 contains 7 faults and version 2 contains 9 faults, respectively. For the two versions, the input vector consists of 28 and 30 inputs, respectively; and the target vector consists of 3 outputs.

Both the subject systems and the associated test cases are developed in java. The test cases are designed and implemented using the junit test framework, in function-level structure. The created versions only involve code modification in order to seed faults in them. The faults are manually seeded based on common mutation operators as mentioned in the literature [40, 41].

4.3. Evaluation methodology

The first step of our approach includes finding the modification related test cases by applying Algorithm 3. Next, to assess the prediction performance of the ANN, we computed the accuracy, precision and recall values for all the three priority classes and averaged them to compute the final outcome, by applying k-fold cross validation. Since, the datasets are small sized, we used k = 5 folds. For unbiased and fair computation of the metrics, the whole dataset is first shuffled and then split into 5-folds, where one fold is kept once for validation, while the remaining 4 folds are used for training. This process is repeated for 5 times based on the number of folds. The accuracy, precision and recall results are stored for each run of each fold and the average of these values are taken as the final result.

Meanwhile, the APFD value has been measured for each fold of each iteration. Finally, the average value is computed. Note that, dataset8 (combination of all the proposed factors) has been used for computing the accuracy, precision, recall and APFD metric. In addition to these metrics, we further estimate the required time to train the ANN model and classify the results in order to determine the applicability of the proposed approach in practice.

To evaluate the effectiveness of our approach, we compared our results with three other approaches: 1. total statement coverage prioritization [10], 2. re-test all and 3. random prioritization. For the prior two approaches, the original test suite has been used. For fair computation, the test suite is divided into the same five folds as Dataset8. In order to prioritize test cases based on total statement coverage approach, we used the feature TC or test cases complexity, since it computes the number of covered statements (requirements) in a test case. The test cases in each fold are then prioritized in descending order according to their covered statements and executed in that order. Similarly, for re-test all approach, the test cases in each fold are executed in their original order. Finally, the APFD value is computed for each fold and the average value is taken as the final result for both these approaches. For the random approach, ten datasets have been created by randomly prioritizing the test cases. The k-fold cross validation is then applied on the datasets. Afterwards, the APFD values are computed according to the aforementioned process. However, we could not perform comparison with other machine learning-based approaches due to the difference of the used artifacts.

Since, we used a number of new attributes for prioritization, it is of great interest to evaluate their significance in the prioritization. To do so, eight datasets are prepared from the test cases for each version of the SUTs. Each of these datasets contain different combination of the prioritization features (subsection 3.1), which are: number of modified modules (MM), test cases complexity (TC), number of modified requirements (MR), modified modules sequence (MMS), modified requirements sequence (MRS), inputs (I) and expected result (E). Since, the inputs and expected result (I, E) are the basic components of a test case, we used these elements in all the combinations of the created datasets. Table 1 presents the combination of factors for each of the datasets.

Table 1
List of datasets containing different factors

Datasets	Factors combinations
Dataset1	{I, E}
Dataset2	{I, E, MM, MMS}
Dataset3	{I, E, TC}
Dataset4	{I, E, MR, MRS}
Dataset5	{I, E, MM, MMS, TC}
Dataset6	{I, E, MR, MRS, TC}
Dataset7	{I, E, MM, MMS, MR, MRS}
Dataset8	{I, E, MM, MMS, MR, MRS, TC}

Total 40 experiments has been performed for each version of the SUTs, i.e. eight experiments for each version. For each of these experiments, the 5-fold cross validation is further employed to compute the classification accuracy. The same number of folds are used for all the eight data sets to make the experiments more comparable.

5. Results and discussion

In this section, we present and discuss the results achieved after evaluating the proposed approach, in terms of the metrics defined in the previous section. Table 2 presents the test suite reduction rate, average accuracy, precision and recall results of the proposed method for each version of the SUTs. The achieved reduction rates for the three versions of the first SUT are 14.42%, 15.38% and 11.53%, respectively and for the two versions of the second SUT are 12.43% and 7.84%, respectively. Here, version 2 of the first SUT achieved the highest value. Though, the achieved reduction rate is not very high, for industry sized systems where there are thousands of test cases, this reduction rate may save hours. Furthermore, our main concern was the prioritization of test cases instead of minimization.

For all the three versions of the first case study, more than 96% of average accuracy was reached. For the same, average precision and recall results were reached up to more than 95%. The second case study also exhibits good results by obtaining average accuracy, precision and recall values up to 98.22%, 97.35% and 98.12%, respectively, for version 1, and 97.11%, 97.23% and 96.73%, respectively, for version 2. The results indicate better prediction quality, which is enough to boost the reliability on the classifier, for future reusability.

Table 3 presents the comparison results of the APFD metric for four prioritization approaches (proposed approach, total statement coverage, re-test all and random prioritization approach). We further split our results for two datasets with and without the proposed factors (i.e. Dataset8 and Dataset1, respectively). The mean APFD values for the proposed approach with Dataset8 and Dataset1, total statement coverage, re-test all and random approach are 0.89, 0.79, 0.88, 0.69 and 0.62, respectively, for case study 1; and 0.83, 0.71, 0.81, 0.61 and 0.56, respectively, for case study 2, which clearly implies the superiority of the proposed approach. The comparison within the ANN based approaches for both datasets further

Table 2
Test suite reduction rate and average accuracy, precision and recall results of the case studies

Case Studies	Versions	Reduction Rate (%)	Accuracy (%)	Precision (%)	Recall (%)
Case Study 1	Version 1	14.42	96.23	94.86	94.74
	Version 2	15.38	98.78	98.90	98.65
	Version 3	11.53	95.39	94.45	93.98
Case Study 2	Version 1	12.43	98.22	97.35	98.12
	Version 2	7.84	97.11	97.23	96.73

Table 3
Average APFD comparison results of various prioritization approaches

Case Studies	Versions	ANN Approach (with Dataset8)	ANN Approach (with Dataset1)	Total Statement Coverage Approach	Re-Test All Approach	Random Approach
Case Study 1	Version 1	0.90	0.81	0.90	0.68	0.68
	Version 2	0.97	0.89	0.94	0.81	0.60
	Version 3	0.80	0.68	0.80	0.59	0.58
Case Study 2	Version 1	0.86	0.75	0.83	0.62	0.60
	Version 2	0.79	0.67	0.78	0.60	0.52

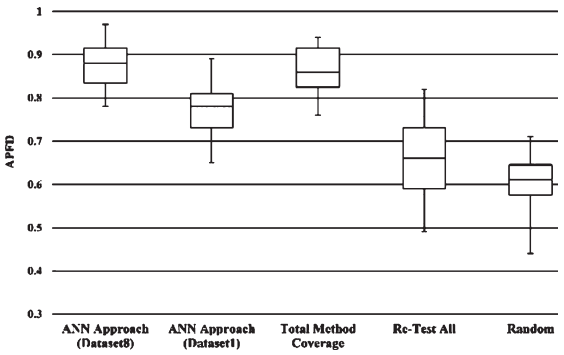

Fig. 2. Box plot of the APFD results of case study 1.

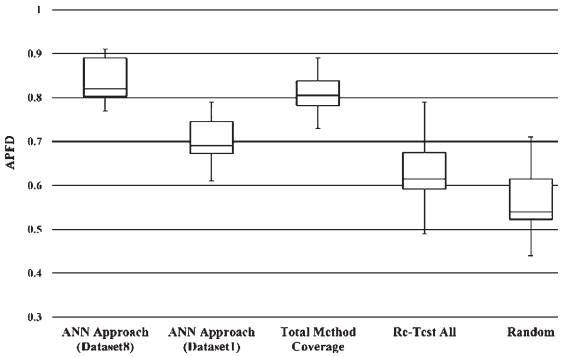

Fig. 3. Box plot of the APFD results of case study 2.

illustrates the fault detection capability of the proposed factors.

In addition, Figs. 2 and 3 represents the results of the APFD metric for different prioritization methods in terms of two box plots. Figure 2, presents the results of the first case study where the proposed approach achieved a median APFD value of 0.88 with a pick value of 0.97. Although, the total statement coverage approach and ANN approach with Dataset1 (without the proposed factors) obtained decent APFD with a pick value of 0.94 and 0.89, respectively, for version 2, still they are outweighed by the proposed approach.

The second case study results, shown in Fig. 3, reveals that the prioritization achieves a median APFD value of 0.81 with and 0.69 without the proposed attributes, and 0.80 for total statement coverage approach. Furthermore, our technique considerably outperforms the re-test all and random prioritization approaches for both the case studies. Hence, the proposed approach provides substantial improvement regarding effectiveness compared to the other prioritization approaches. Therefore, the results indicate that the proposed approach is able to find effective test cases which are important for a particular software version.

In order to determine the time effectiveness of our approach, we computed the percentage of the executed test cases required for detecting the seeded faults for all the four prioritization approaches. The results show that our approach is able to detect faults by executing only 22% of the total test cases whereas total statement coverage, re-test all and random approach requires 24%, 47% and 59%, respectively, for the first case study. Similarly, for the second case study, they require 53%, 61%, 69% and 73% of the total test cases, respectively. Therefore, in both cases our proposed ANN based approach can reveal faults early than the other three approaches, by executing minimum number of test cases. Thus, the time

Table 4
Average classification accuracy of the case studies for different datasets (%)

Datasets	Case Study 1			Case Study 2	
	Version 1	Version 2	Version 3	Version 1	Version 2
Dataset1	82.61	74.47	59.80	87.64	87.73
Dataset2	86.29	90.15	89.64	87.34	85.62
Dataset3	83.42	91.47	75.42	86.62	79.35
Dataset4	89.66	90.96	86.36	89.89	91.99
Dataset5	91.40	89.97	86.85	90.55	86.43
Dataset6	90.07	92.17	90.69	95.62	95.78
Dataset7	92.45	97.41	89.59	94.33	98.23
Dataset8	94.87	98.78	92.39	98.22	97.16

Table 5
Total required time for training and classification

Case Studies	Versions	Datasets	Training Time (ms)	Classification Time (ms)
Case Study 1	Version 1	Dataset8	64.11	1.12
		Dataset1	39.30	0.99
	Version 2	Dataset8	65.04	1.06
		Dataset1	32.54	0.82
	Version 3	Dataset8	62.77	0.91
		Dataset1	34.21	0.78
Case Study 2	Version 1	Dataset8	31.65	0.31
		Dataset1	36.15	0.34
	Version 2	Dataset8	36.37	0.36
		Dataset1	35.69	0.32

required for testing will be reduced correspondingly, increasing the time effectiveness.

Furthermore, Table 4 presents the accuracies obtained after feeding the prepared datasets (Table 1) to the ANN model, in order to evaluate the significance of the factors in the classification accuracy. For all the three versions of the credit card approval system, high accuracies (94.87%, 98.78% and 92.39%, respectively) were obtained for the combination of all the proposed factors (Dataset8). On the other hand, low accuracies (82.61%, 74.47% and 59.80%, respectively) were achieved for Dataset1 (containing no proposed factors).

For the second case study, version 1 achieved high accuracy (98.22%) with Dataset8, which version 2 achieved for Dataset7 (98.23%). Since, Dataset7 and Dataset8 differs from each other for only one feature (i.e. TC) and the difference within the accuracies are negligible, we can consider them as identical. Here, low accuracies (86.62% and 79.35%, respectively) were obtained for Dataset3, which indicate that the attribute TC has slightly less contributed to the prioritization, whereas highest contributed features were MM and MR.

Accuracy is the measurement metric of the performance of the classifier i.e., better accuracy indicates more accurate prioritization (prediction). Thus, high accuracy is related to high fault detection rate and low accuracy indicates low fault detection rate. In summary, the prioritization achieved by Dataset8 with high accuracies are more likely to reveal faults, whereas Dataset1 with relatively low accuracies is less likely to reveal faults in the regression testing.

Moreover, the applicability of the proposed approach is assessed for Dataset8 and Dataset1 in terms of the training and classification time, shown in Table 5. The results demonstrate that the training times with Dataset8 is slightly higher than that of Dataset1. This is because of the difference of various features in the datasets, i.e. Dataset8 containing more features than Dataset1. The times required for classification also indicate a negligible difference for both datasets, all of which are in the range of milliseconds. Therefore, we claim that the proposed approach is fairly applicable and feasible.

5.1. Threats to validity

Construct validity. Threats to construct validity involves the measurement metrics used in the empirical investigations. The metric chosen in this study for finding the fault detection rate, such as APFD has alternative metrics, use of which may affect the outcome of the study. However, this metric has been chosen due to its preferences in previous studies. Other metrics such as accuracy, precision and recall have also been selected based on past machine learning based researches, though, we used them from a regression testing perspective.

Internal validity. Such threats may affect the outcomes of the empirical studies. As a result, the effectiveness of the prioritization approach could be threatened. The implementation of the proposed algorithms may contain potential faults. To limit this threat, the results of several simple java programs have been validated manually.

External validity. The threats to external validity includes two concerns. The first concern involves the subject projects. In this research, the empirical studies have been performed on two medium sized projects, while, large industrial projects with huge requirements and altered characteristics may lead to different outcome. The second concern involves the use of seeded faults which may hinder the generalization of the results. Conducting experiments with larger industrial projects and real faults can be considered as a future direction for controlling these threats.

6. Conclusion

This paper presents a version specific test case prioritization approach by applying artificial neural networks with the intension to improve fault detection rate and cost effectiveness. Three new factors have been proposed based on which the neural network is able to prioritize new test cases. The proposed approach is validated through two phases of empirical studies with two case studies. In these two phases, the reliability on the ANN classifier and the fault detection capability of the proposed approach have been evaluated. Results of the proposed approach indicates better reliability by obtaining an average of more than 97% accuracy and improved fault detection rate compared to existing prioritization approaches. Besides, selecting only the version specific test cases adds a new dimension to the approach in terms of cost effectiveness.

Future direction includes the evaluation of the proposed approach with large-scale industrial projects containing real faults. We also plan to apply other machine learning techniques such as SVM or Info-Fuzzy Networks, and perform comparison within the outcomes of these techniques in order to assess the most effective one. In addition, applying an ensemble method (ex. boosting) [33] which reuses the classification models from the previous learning phases to further amplify the classification quality is also an auspicious future direction.

References

[1] L. Tahat, B. Korel, G. Koutsogiannakis and N. Almasri, State-based models in regression test suite prioritization, *Software Quality Journal* **25** (2016), 703–742.

[2] C. Zhang, Z. Chen, Z. Zhao, S. Yan, J. Zhang and B. Xu, An improved regression test selection technique by clustering execution profiles, *Proceedings of the 10th IEEE International Conference on Quality Software (QSIC)*, 2010, pp. 171–179.

[3] C.-T. Lin, K.-W. Tang, J.S. Wang and G.M. Kapfhammer, Empirically evaluating Greedy based test suite reduction methods at different levels of test suite complexity, *Science of Computer Programming* **150** (2017), 1–25.

[4] S. Yoo and M. Harman, Regression testing minimization, selection and prioritization: A survey, *Software Testing, Verification and Reliability* **22**(2) (2012), 67–120.

[5] S. Elbaum, A.G. Malishevsky and G. Rothermel, Incorporating varying test costs and fault severities into test case prioritization, *Proceedings of the 23rd IEEE International Conference on Software Engineering, (ICSE)*, 2001, pp. 329–338.

[6] R.H. Rosero, O.S. Gomez and G. Rodriguez, Regression testing of database applications under an incremental software development setting, *IEEE Access* **5** (2017), 18419–18428.

[7] C.-T. Lin, K.-W. Tang and G.M. Kapfhammer, Test suite reduction methods that decrease regression testing costs by identifying irreplaceable tests, *Software Quality Journal* **56**(10) (2016), 1322–1344.

[8] C. Catal and D. Mishra, Test case prioritization: A systematic mapping study, *Software Quality Journal* **21** (2012), 445–478.

[9] H. Park, H. Ryu and J. Baik, Historical value-based approach for cost-cognizant test case prioritization to improve the effectiveness of regression testing, *Proceedings of the 2nd IEEE International Conference on Secure System Integration and Reliability Improvement*, 2008, pp. 39–46.

[10] S. Elbaum, A.G. Malishevsky and G. Rothermel, Test case prioritization: A family of empirical studies, *IEEE Transactions on Software Engineering* **28**(2) (2002), 159–182.

[11] R. Krishnamoorthi and S.A.S.A. Mary, Factor oriented requirement coverage based system test case prioritization of new and regression test cases, *Information and Software Technology* **51**(4) (2009), 799–808.

[12] Y.T. Yu and M.F. Lao, Fault-based test suite prioritization for specification-based testing, *Information and Software Technology* **54** (2012), 179–202.

[13] P. Ammann and J. Offutt, Introduction to software testing, Cambridge University Press, 2nd edition, 2008.

[14] D.D. Nardo, N. Alshahwan, L. Briand and Y. Labiche, Coverage-based regression test case selection, minimization and prioritization: A case study on an industrial system, *Software Testing Verification and Reliability* **25**(4) (2015), 371–396.

[15] J.A. Jones and M.J. Harrold, Test-suite reduction and prioritization for modified condition/decision coverage, *IEEE Transactions on Software Engineering* **29**(3) (2003), 195–209.

[16] H. Srikanth, C. Hettiarachchi and H. Do, Requirements based test prioritization using risk factors: An industrial study, *Information and Software Technology* **69** (2016), 71–83.

[17] C. Hettiarachchi, H. Do and B. Choi, Effective regression testing using requirements and risks, *Proceedings of the 8th IEEE International Conference on Software Security and Reliability (SERE)*, 2014, pp. 157–166.

[18] B. Qu, C. Nie and B. Xu, Test case prioritization for black box testing, *Proceedings of the 31st Annual International Computer Software Applications Conference (COMPSAC)*, 2007, pp. 465–474.

[19] H. Srikanth, M. Cashman and M.B. Cohen, Test case prioritization of build acceptance tests for an enterprise cloud application: An industrial case study, *Journal of Systems and Software* **119** (2016), 122–135.

[20] E. Engstrom, P. Runeson and A. Ljung, Improving regression testing transparency and efficiency with history-based prioritization-an industrial case study, *Proceedings of the 4th IEEE International Conference on Software Testing, Verification and Validation*, 2011, pp. 367–376.

[21] A. Schwartza and H. Do, Cost-effective regression testing through adaptive test prioritization strategies, *The Journal of Systems and Software* **115** (2016), 61–81.

[22] A.R. Lenz, A. Pozo and S.R. Vergilio, Linking software testing results with a machine learning approach, *Engineering Applications of Artificial Intelligence* **26**(5-6) (2013), 1631–1640.

[23] N. Gökçe, M. Eminov and F. Belli, Coverage-based, prioritized testing using neural network clustering, In: A. Levi, et al. (eds.) *Computer and Information Sciences – ISCIS. Lecture Notes in Computer Science*, Springer, Belin-Heidelberg, 4263, 2006, pp. 1060–1071.

[24] M.J. Arafeen and H. Do, Test case prioritization using requirements-based clustering, *Proceedings of the 6th IEEE International Conference on Software Testing, Verification and Validation*, 2013, pp. 312–321.

[25] M.A. Hasan, M.A. Rahman and M.S. Siddik, Test case prioritization based on dissimilarity clustering using historical data analysis, In: S. Kaushik, et al. (eds.) *Information, Communication and Computing Technology- ICICCT. Communications in Computer and Information Science*, Springer, Singapore, 750, 2017, pp. 269–281.

[26] R. Lachmann, S. Schulze, M. Nieke and I. Schaefer, System-level test case prioritization using machine learning, *Proceedings of the 15th IEEE International Conference on Machine Learning Applications (ICMLA)*, 2016, pp. 361–368.

[27] H. Bhasin and E. Khanna, Neural network based black box testing, *SIGSOFT Software Engineering Notes* **39**(2) (2014), 1–6.

[28] F. Wang, S.C. Yang and Y.L. Yang, Regression testing based on neural networks and program slicing techniques, In: Y. Wang and T. Li, (eds.) *Practical Applications of Intelligent Systems. Advances in Intelligent and Soft Computing*, Springer, Belin-Heidelberg, 124, 2011, pp. 409–418.

[29] S. Mirarab and L. Tahvildari, An empirical study on Bayesian network-based approach for test case prioritization, *Proceedings of the 1st IEEE International Conference on Software Testing, Verification and Validation*, 2008, pp. 278–287.

[30] C. Hettiarachchi, H. Do and B. Choi, Risk-based test case prioritization using a fuzzy expert system, *Information and Software Technology* **69** (2016), 1–15.

[31] G.Y.-H. Chen and P.-Q. Wang, Test case prioritization in a specification-based testing environment, *Journal of Software* **9**(8) (2014), 2056–2064.

[32] S.F. Ahmad, D.K. Singh and P. Suman, Prioritization for regression testing using Ant Colony Optimization based on test factors, In: R. Singh, et al. (eds.) *Intelligent Communication, Control and Devices. Advances in Intelligent Systems and Computing*, Springer, Singapore, 624, 2018, pp. 1353–1360.

[33] A. Perini, A. Susi and P. Avesani, A machine learning approach to software requirements prioritization, *IEEE Transactions on Software Engineering* **39**(4) (2013), 445–461.

[34] H. Jahan, S. Rao and D. Liu, Test case generation for BPEL-based web service composition using Colored Petri Nets, *Proceedings of IEEE International Conference on Progress in Informatics and Computing (PIC)*, 2016, pp. 623–628.

[35] S. Rao, H. Jahan and D. Liu, A search-based approach for test suite generation from Extended Finite State Machines, *Proceedings of IEEE International Conference on Progress in Informatics and Computing (PIC)*, 2016, pp. 82–87.

[36] I.H. Witten, E. Frank and M.A. Hall, Data mining: Practical machine learning tools and techniques, Morgan Kaufmann Publishers Inc., 3rd edition, 2011.

[37] S. Wang and X. Yao, Multiclass imbalance problems: Analysis and potential solutions, *IEEE Transactions on Systems, Man, and Cybernetics* **42**(4) (2012), 1119–1130.

[38] M.H. Hassoun, *Fundamentals of Artificial Neural Networks*, MIT Press, 1995.

[39] S. Pal and S. Gulli, *Deep Learning with Keras: Implementing deep learning models and neural networks with the power of Python*, Packt Publishing, 2017.

[40] M. Vanmali, M. Last and A. Kandel, Using a neural network in the software testing process, *International Journal of Intelligent Systems* **17**(1) (2002), 45–62.

[41] S.R. Shahamiri, W.M.N. Wan-Kadir, S. Ibrahim and S.Z.M. Hashim, Artificial neural networks as multi-networks automated test oracle, *Automated Software Engineering* **19** (2012), 303–334.